VEHICLE RENTAL BOOKING SYSTEM

CS23531 – WEB PROGRAMMING
MINI PROJECT REPORT

Submitted by

GEEVAN J V – 2116241001061

Of

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM

(An Autonomous Institution)



NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "VEHICLE RENTAL BOOKING SYSTEM" is the bonafide work of GEEVAN JV (2116241001061) who carried out the work under my supervision.

SIGNATURE
Dr. P. Neelaveni
HEAD OF THE DEPARTMENT
Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam,
Chennai - 602105

SIGNATURE
Mrs. Janani M
COURSE INCHARGE
Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam,
Chennai - 602105

This project is submitted for CS23531 – Web Programming held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

The Vehicle Rental Booking System is a web-based application developed using PHP, MySQL and Bootstrap. It allows customers to browse the available cars, bikes and other vehicles of a rental agency, check availability for chosen dates, book a vehicle online and track their bookings, while giving the administrator a single place to manage the fleet, customers and reservations. The application follows a client-server architecture in which Bootstrap-based pages are served by PHP scripts running on an Apache server, and all users, vehicles, bookings and payments are stored in a MySQL database.

Customers can register and log in, search vehicles by type, brand and price, select pick-up and return dates, view the automatically calculated rental cost, and cancel a booking before it is approved. The administrator can add, edit and delete vehicles, approve or reject bookings, view customer details and see booking reports. The main objective of the system is to replace manual registers and phone-based reservations with a simple, responsive and reliable online process that avoids double bookings and saves time for both customers and the rental agency. Tagline: Rent a Vehicle. Drive Your Way.

TABLE OF CONTENTS

S.No	Section	Title
1	1.1	Introduction
2	1.2	Purpose
3	1.3	Scope of the Project
4	1.4	Software Requirements Specification
5	2.1	Use Case Diagram
6	2.2	Entity Relationship Diagram
7	2.3	Data Flow Diagram
8	3.	Module Description
9	4.1	User Interface and Workflow
10	4.2	Data Design
11	4.3	Selected Source Code
12	4.4	Testing Scenarios
13	5.	Conclusion
14	6.	References

1. PROJECT OVERVIEW

1.1 INTRODUCTION

Many small and medium vehicle rental agencies still record bookings in paper registers or on phone calls. This makes it difficult to know which vehicles are free on a given date, leads to double bookings, and forces customers to visit or call the office to confirm even a simple reservation. The Vehicle Rental Booking System provides a centralised web portal where customers can view the complete fleet with rates and photos, choose dates and book a vehicle in a few clicks, while the administrator keeps the fleet and reservations up to date from a dashboard. The application is implemented with PHP on the server side, MySQL for data storage and the Bootstrap framework for a responsive user interface.

1.2 PURPOSE

- Replace manual registers and phone bookings with an organised online booking process.
- Let customers search, compare and book vehicles at any time from a laptop or mobile browser.
- Prevent double bookings by checking date availability before a reservation is accepted.
- Give the administrator simple tools to manage vehicles, customers and bookings.
- Calculate rental cost automatically from the daily rate and the number of days.

1.3 SCOPE OF THE PROJECT

- Customer registration, login and profile management.
- Vehicle catalogue with category-wise listing, search and filters.
- Online booking with date selection, availability check and automatic cost calculation.
- Booking history, cancellation of pending bookings and payment record.
- Admin panel to manage vehicles, bookings, customers and reports.

Limitations: The system does not integrate a live payment gateway; payment is recorded as cash or pay-at-pickup, or entered by the administrator. It is intended for a single rental agency, does not include GPS tracking or driver management, and email or SMS notifications are not part of the current version.

1.4 SOFTWARE REQUIREMENTS SPECIFICATION

Architecture: client-server web application. The browser sends HTTP requests to PHP scripts hosted on an Apache server. The scripts validate the session, apply the business rules, read or write the MySQL database using prepared statements and return HTML pages styled with Bootstrap.

Hardware requirements

- Processor: dual-core or higher (suggested)
- RAM: 4 GB minimum, 8 GB recommended to run Apache, MySQL, a browser and an editor together
- Display: standard laptop or desktop

Software requirements

- Windows, macOS or Linux with a modern web browser (Chrome, Edge or Firefox)
- XAMPP or WAMP stack providing Apache 2.4, PHP 8.x and MySQL 8.x (or MariaDB)
- Bootstrap 5 with HTML5, CSS3 and JavaScript (ES6)
- phpMyAdmin for database management
- Visual Studio Code or an equivalent editor, with Postman or a browser for testing

Functional Requirements

- **Authentication:** customers register and log in; the admin logs in through a separate role.
- **Vehicle management:** admin can add, edit, delete and mark vehicles as available or unavailable.
- **Search:** filter vehicles by type, brand, price and availability dates.
- **Booking:** select dates, check availability, view total cost and confirm the booking.
- **Booking management:** admin approves or rejects bookings; customers can view and cancel pending ones.
- **Payment record:** store the amount, method and date for every booking.
- **Reports:** admin can view bookings and revenue summaries.

Non-functional Requirements

- Simple, responsive interface that works on laptops and mobile browsers.
- Secure storage of passwords using password_hash and protection against SQL injection using prepared statements.
- Fast page loads and reliable data storage with consistent date validation.
- Clear modular code structure that is easy to extend.

2. SYSTEM FLOW DIAGRAMS

2.1 USE CASE DIAGRAM

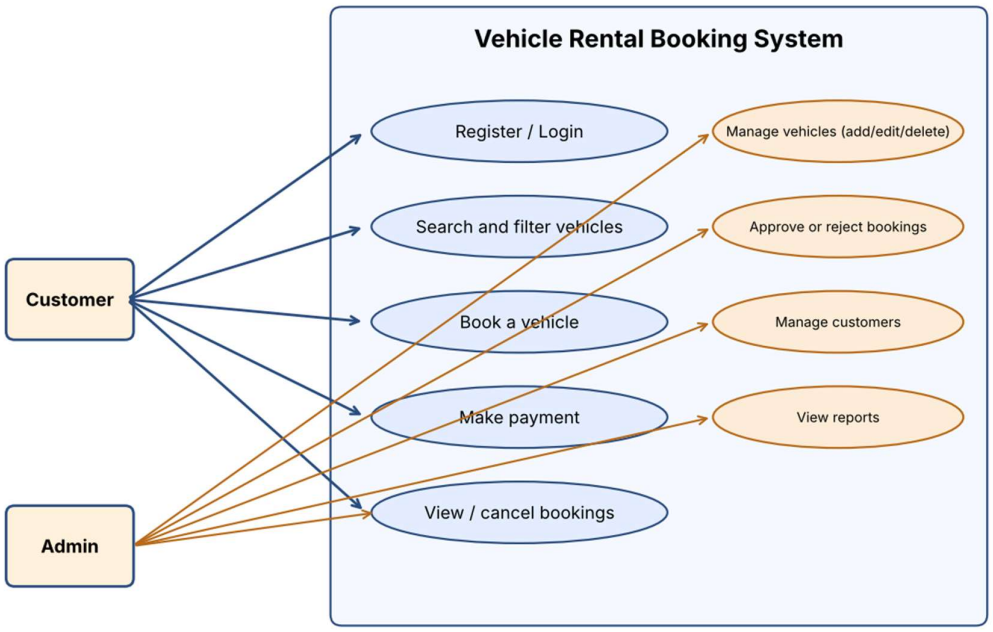


Figure 2.1: Customer and admin use cases

2.2 ENTITY RELATIONSHIP DIAGRAM

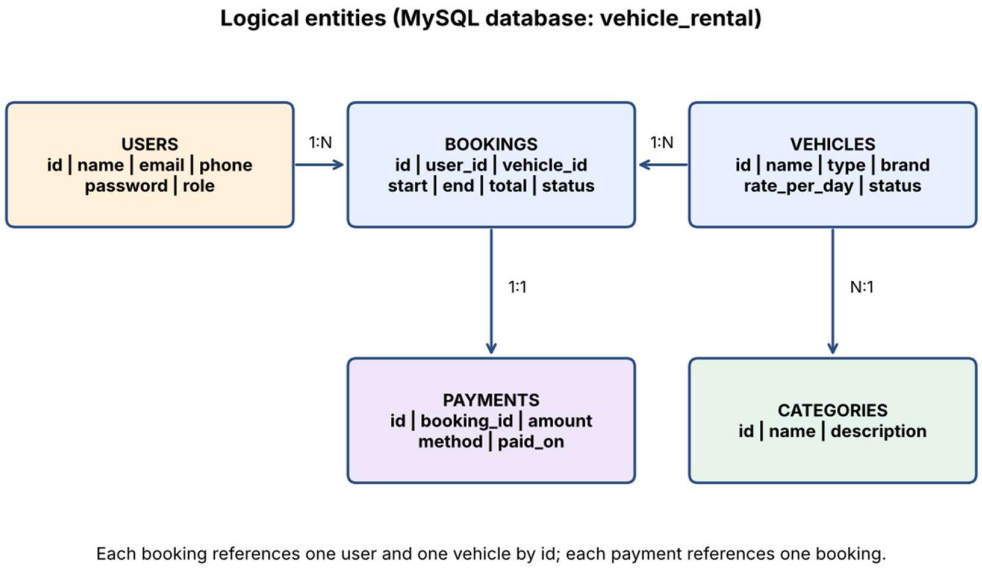


Figure 2.2: Main entities stored in the MySQL database

2.3 DATA FLOW DIAGRAM

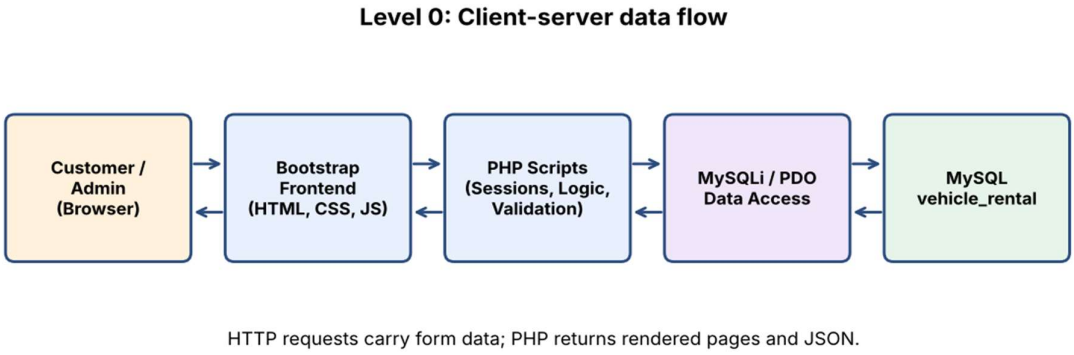


Figure 2.3: Browser, Bootstrap frontend, PHP scripts and database data flow

3. MODULE DESCRIPTION

- **Authentication Module:** customer registration and login, password hashing, session handling and role-based access for admin and customer.
- **Vehicle Module:** add, edit, delete and list vehicles with category, brand, daily rate, image and availability status.
- **Search and Filter Module:** search by keyword and filter by type, brand, price range and available dates.
- **Booking Module:** date selection, availability check, cost calculation and creation of a booking with status pending, approved, rejected or cancelled.
- **Payment Module:** records the amount, payment method and date for each booking.
- **Customer Module:** profile view and update, booking history and cancellation of pending bookings.
- **Admin Module:** dashboard with counts of vehicles, customers and bookings, booking approval and customer management.
- **Report Module:** booking and revenue summaries by date range.

4. IMPLEMENTATION

4.1 USER INTERFACE AND WORKFLOW

Step 1: Install XAMPP, start Apache and MySQL, copy the project folder into htdocs and import the vehicle_rental.sql file using phpMyAdmin. Step 2: Set the database host, user and password in config.php and open http://localhost/vehicle_rental/. Step 3: Register as a customer and log in. Step 4: Browse or search vehicles and open a vehicle to view its details. Step 5: Choose the pick-up and return dates, check the total cost and confirm the booking. Step 6: Track the booking under My Bookings. Step 7: The admin logs in to the dashboard, adds or edits vehicles, approves or rejects bookings and views reports.

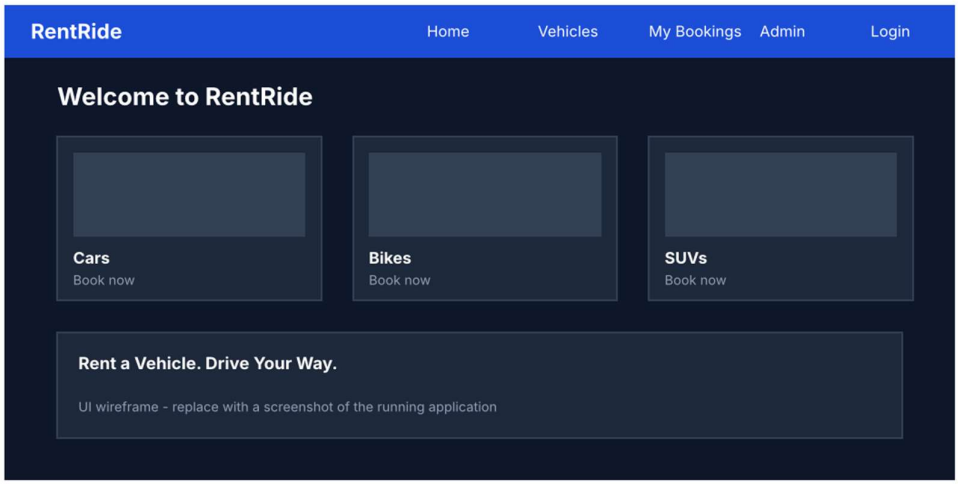


Figure 4.1: Home page

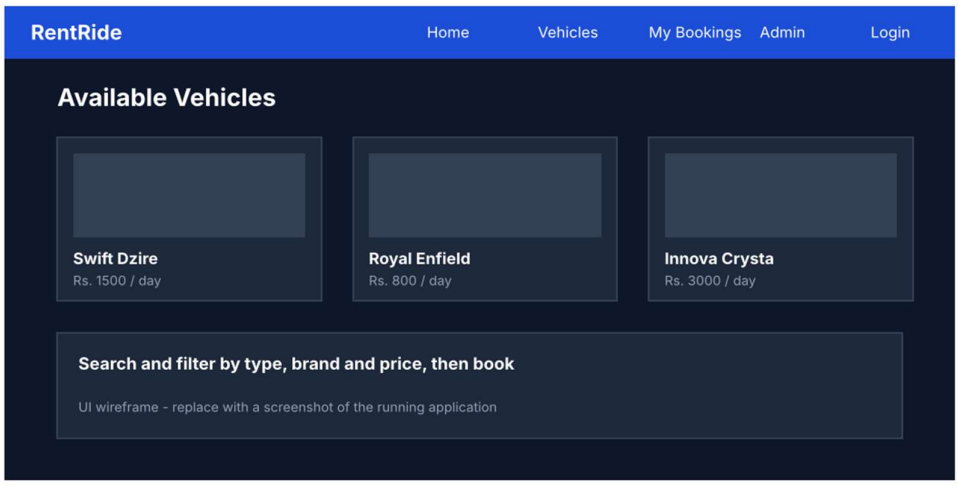


Figure 4.2: Vehicle listing with search and filters

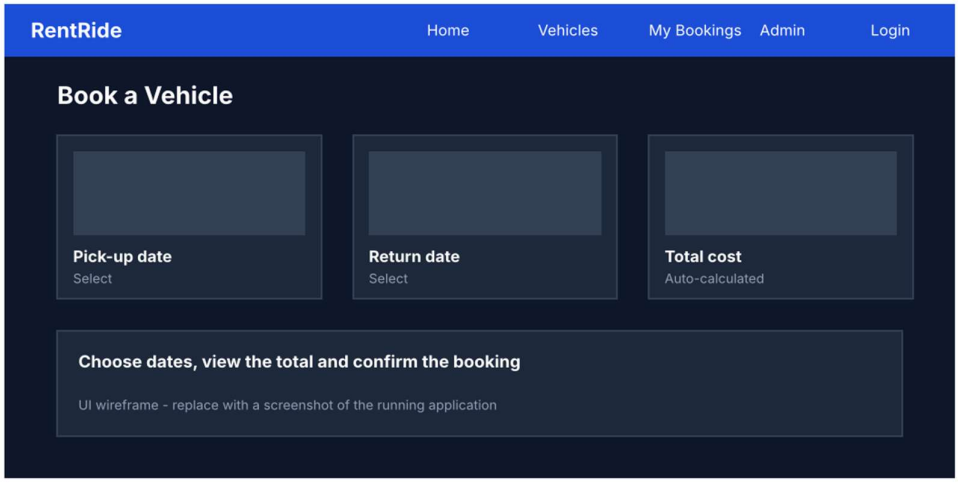


Figure 4.3: Booking form with automatic cost calculation

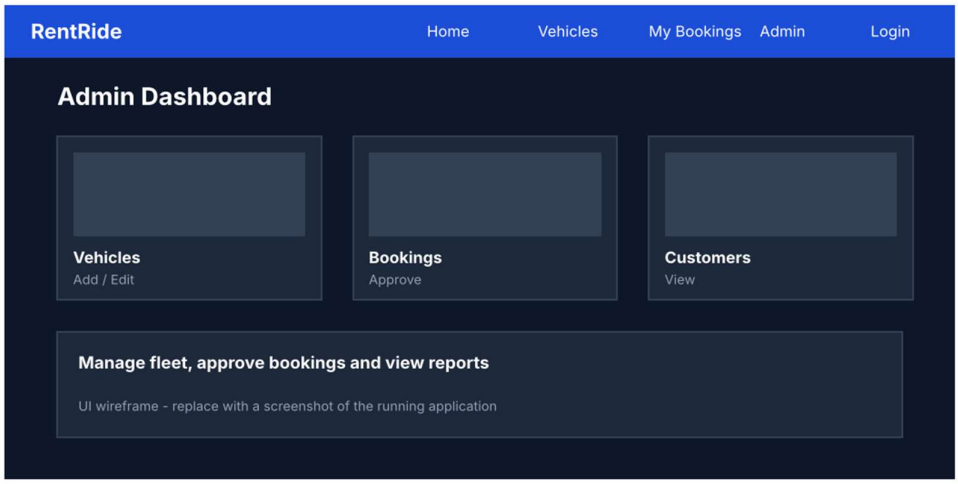


Figure 4.4: Admin dashboard

4.2 DATA DESIGN

The application stores its data in a MySQL database named vehicle_rental, which holds the tables users, categories, vehicles, bookings and payments. Each table uses an auto-incremented primary key, and the bookings table references users and vehicles through foreign keys so that every booking is linked to one customer and one vehicle.

Table	Key columns	Purpose
users	id, name, email, phone, password, role	Customers and admin accounts
categories	id, name, description	Vehicle types such as Car, Bike, SUV
vehicles	id, category_id, name, brand, rate_per_day, image, status	Fleet details and availability
bookings	id, user_id, vehicle_id, start_date, end_date, total, status	Reservations made by customers

Table	Key columns	Purpose
payments	id, booking_id, amount, method, paid_on	Payment record for each booking

Formula: Total cost = rate_per_day x number of days, where number of days = (return date - pick-up date) + 1, and the return date must not be earlier than the pick-up date. A vehicle is available for a requested period only if no approved or pending booking overlaps it, that is, no existing booking has start_date <= requested end_date and end_date >= requested start_date.

4.3 SELECTED SOURCE CODE

1. Database schema (vehicle_rental.sql)

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(80) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  phone VARCHAR(15),
  password VARCHAR(255) NOT NULL,
  role ENUM('customer','admin') DEFAULT 'customer'
);
CREATE TABLE vehicles (
  id INT AUTO_INCREMENT PRIMARY KEY,
  category_id INT,
  name VARCHAR(100) NOT NULL,
  brand VARCHAR(50),
  rate_per_day DECIMAL(10,2) NOT NULL,
  image VARCHAR(255),
  status ENUM('available','unavailable') DEFAULT 'available'
);
CREATE TABLE bookings (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  vehicle_id INT NOT NULL,
  start_date DATE NOT NULL,
  end_date DATE NOT NULL,
  total DECIMAL(10,2),
  status ENUM('pending','approved','rejected','cancelled') DEFAULT 'pending',
  FOREIGN KEY (user_id) REFERENCES users(id),
  FOREIGN KEY (vehicle_id) REFERENCES vehicles(id)
);
```

2. Database connection (config.php)

```
<?php
session_start();
$conn = new mysqli("localhost", "root", "", "vehicle_rental");
if ($conn->connect_error) {
    die("Database connection failed");
}
?>
```

3. Customer registration (register.php)

```
<?php
require "config.php";
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $name  = trim($_POST["name"]);
    $email = trim($_POST["email"]);
    $hash  = password_hash($_POST["password"], PASSWORD_DEFAULT);
    $stmt = $conn->prepare(
      "INSERT INTO users (name, email, password) VALUES (?, ?, ?)");
    $stmt->bind_param("sss", $name, $email, $hash);
    if ($stmt->execute()) { header("Location: login.php"); exit; }
```

```
    $error = "Email already registered";
}
?>
```

4. Login (login.php)

```
<?php
require "config.php";
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $stmt = $conn->prepare("SELECT * FROM users WHERE email = ?");
    $stmt->bind_param("s", $_POST["email"]);
    $stmt->execute();
    $user = $stmt->get_result()->fetch_assoc();
    if ($user && password_verify($_POST["password"], $user["password"])) {
        $_SESSION["uid"]  = $user["id"];
        $_SESSION["role"] = $user["role"];
        header("Location: " . ($user["role"] === "admin"
               ? "admin/index.php" : "vehicles.php"));
        exit;
    }
    $error = "Invalid email or password";
}
?>
```

5. Vehicle search with filters (vehicles.php)

```
<?php
require "config.php";
$q    = "%" . ($_GET["q"] ?? "") . "%";
$type = (int)($_GET["type"] ?? 0);
$sql  = "SELECT * FROM vehicles WHERE status='available' AND name LIKE ?";
if ($type) { $sql .= " AND category_id = " . $type; }
$stmt = $conn->prepare($sql . " ORDER BY rate_per_day");
$stmt->bind_param("s", $q);
$stmt->execute();
$rows = $stmt->get_result();
?>
<div class="row">
<?php while ($v = $rows->fetch_assoc()) { ?>
  <div class="col-md-4 mb-4">
    <div class="card h-100">
      <div class="card-body">
        <h5 class="card-title"><?= htmlspecialchars($v["name"]) ?></h5>
        <p>Rs. <?= $v["rate_per_day"] ?> / day</p>
        <a class="btn btn-primary" href="book.php?id=<?= $v["id"] ?>">Book</a>
      </div>
    </div>
  </div>
<?php } ?>
</div>
```

6. Booking with availability check (book.php)

```
<?php
require "config.php";
if (!isset($_SESSION["uid"])) { header("Location: login.php"); exit; }
$vid   = (int)$_POST["vehicle_id"];
$start = $_POST["start_date"];
$end   = $_POST["end_date"];
if ($end < $start) { die("Return date must be after pick-up date"); }

$chk = $conn->prepare("SELECT COUNT(*) FROM bookings
    WHERE vehicle_id = ? AND status IN ('pending','approved')
    AND start_date <= ? AND end_date >= ?");
$chk->bind_param("iss", $vid, $end, $start);
$chk->execute();
$chk->bind_result($clash);
$chk->fetch();
$chk->close();
if ($clash > 0) { die("Vehicle not available for these dates"); }
```

```
$rate = $conn->query("SELECT rate_per_day FROM vehicles
                      WHERE id = $vid")->fetch_row()[0];
$days  = (strtotime($end) - strtotime($start)) / 86400 + 1;
$total = $rate * $days;

$ins = $conn->prepare("INSERT INTO bookings
    (user_id, vehicle_id, start_date, end_date, total)
    VALUES (?, ?, ?, ?, ?)");
$ins->bind_param("iissd", $_SESSION["uid"], $vid, $start, $end, $total);
$ins->execute();
header("Location: my_bookings.php");
?>
```

7. Admin approval of a booking (admin/approve.php)

```
<?php
require "../config.php";
if (($_SESSION["role"] ?? "") !== "admin") { die("Access denied"); }
$id     = (int)$_GET["id"];
$status = $_GET["action"] === "approve" ? "approved" : "rejected";
$stmt = $conn->prepare("UPDATE bookings SET status = ? WHERE id = ?");
$stmt->bind_param("si", $status, $id);
$stmt->execute();
header("Location: bookings.php");
?>
```

8. Bootstrap booking form with cost preview (JavaScript)

```
<form method="post" action="book.php" class="row g-3">
  <input type="hidden" name="vehicle_id" value="<?= $v["id"] ?>">
  <div class="col-md-6">
    <label class="form-label">Pick-up date</label>
    <input type="date" name="start_date" id="start" class="form-control" required>
  </div>
  <div class="col-md-6">
    <label class="form-label">Return date</label>
    <input type="date" name="end_date" id="end" class="form-control" required>
  </div>
  <p>Total: Rs. <span id="total">0</span></p>
  <button class="btn btn-success">Confirm Booking</button>
</form>
<script>
const rate = <?= $v["rate_per_day"] ?>;
function calc() {
  const s = new Date(start.value), e = new Date(end.value);
  const d = Math.round((e - s) / 86400000) + 1;
  total.textContent = d > 0 ? d * rate : 0;
}
start.onchange = end.onchange = calc;
</script>
```

4.4 TESTING SCENARIOS

Test	Action	Expected result
T1	Register with valid details	Account is created and the user can log in
T2	Register with an existing email	Registration is rejected with a message
T3	Log in with a wrong password	Login fails with an error message
T4	Search vehicles by name and type	Only matching available vehicles are shown
T5	Book a vehicle for free dates	Booking is saved with status pending and the correct total
T6	Book a vehicle for overlapping dates	Booking is rejected as the vehicle is not available

Test	Action	Expected result
T7	Select a return date before the pick-up date	Booking is rejected with a date error
T8	Admin approves or rejects a booking	Status is updated and shown to the customer
T9	Customer opens an admin page	Access is denied
T10	Admin adds, edits and deletes a vehicle	Changes appear in the vehicle listing

These scenarios are intended to be verified manually on the running application, along with checks of the database records in phpMyAdmin.

5. CONCLUSION

The Vehicle Rental Booking System provides a practical online platform where customers can search vehicles, check availability, book for chosen dates and track their reservations, while the administrator manages the fleet and approvals from one dashboard. By combining PHP session-based authentication, a MySQL database with an availability check, and a responsive Bootstrap interface, the project removes the need for manual registers and avoids double bookings. Future extensions could include an online payment gateway, email and SMS notifications, vehicle image galleries, driver management, GPS tracking, customer reviews and ratings, and automated testing.

6. REFERENCES

- PHP Manual - https://www.php.net/manual/en/
- MySQL Reference Manual - https://dev.mysql.com/doc/
- Bootstrap Documentation - https://getbootstrap.com/docs/
- MDN Web Docs (HTML, CSS, JavaScript) - https://developer.mozilla.org/
- XAMPP Apache Friends - https://www.apachefriends.org/
- Project source: Vehicle Rental Booking System web application (PHP scripts, SQL schema, Bootstrap pages)